

设计报告

赛题分析

APSP

此赛题要求我们在单张 GPU 上，利用 HIP 编程模型实现一个高效的全源最短路径（APSP）求解器。需要我们编写自定义算法，读取一个包含图信息的输入文件，并在处理大规模顶点数（最高 40,000）的图后，以特定的格式将计算出的距离矩阵输出到标准输出。整个实现的核心挑战在于如何将经典的 APSP 算法（如分块 Floyd-Warshall）高效地并行化，充分利用 GPU 的计算能力和共享内存来优化性能，从而在满足严格的输入输出格式要求的同时，解决大规模图的路径计算问题。

prefix_sum

此赛题要求我们在单个 GPU 上，利用 HIP 编程模型实现一个高效的前缀和求解器。需要我们编写自定义的 GPU 核函数，处理一个包含大量整数的输入数组，并以特定的格式将计算出的前缀和结果输出到标准输出。整个实现的核心挑战在于如何将经典的并行前缀和算法有效地映射到 GPU 上，通过多阶段的并行计算来处理大规模数据，从而在满足严格的输入输出格式要求的同时，确保优异的性能。

Softmax

此赛题要求我们在单个 GPU 上，利用 HIP 编程模型实现一个高效且数值稳定的 Softmax 函数。需要我们处理一个包含大量浮点数的输入数组，并遵循赛题提供的数值稳定公式，计算出对应的 Softmax 值。整个实现的核心挑战在于如何将分阶段的 Softmax 计算过程（寻找最大值、指数化、求和与归一化）高效地并行化，特别是在处理大规模数据时，需要运用 GPU 编程技巧来避免数据传输瓶颈和数值溢出问题，最终将满足特定精度的结果输出到标准输出。

优化操作

APSP

我们实现了分块的 Floyd-Warshall 算法，核心优化思路在于将大规模的距离矩阵划分为多个子块，并通过三个阶段的内核操作，利用 GPU 的共享内存来最大化数据局部性并减少对慢速全局内存的访问。在每个主循环步长中，代码首先更新一个主对角线上的“枢纽块”，然后利用这个更新后的枢纽块并行更新与其同行的所有块和同列的所有块，最后再并行更新所有剩余的块。这种分阶段、分块的并行策略允许线程块在处理子矩阵时将所需数据加载到快速的共享内存中，从而实现高效的计算和数据重用，最终显著提高了大规模图算法的性能。

prefix_sum

我们通过分层递归的策略，实现了对超大规模数组的高效并行前缀和计算。核心优化思路是将整个前缀和问题分解为多个子问题：首先在每个线程块内使用基于 `__shfl_up`（高效的 warp 间通信）的并行扫描算法快速计算局部前缀和；然后将每个块的总和提取出来，对这些“块总和”数组进行递归地前缀和计算，直到数组大小足够小，可以在单个块内处理；最后，将计算出的块偏移量加回到每个线程块的局部前缀和结果中，以得到最终的全局前缀和。我们利用了 GPU 的不同内存层次（寄存器、共享内存和全局内存），并通过递归处理任意大小的输入，通过这种多阶段、多层次的并行化方法，有效地解决了单次内核启动无法处理海量数据的挑战，实现了兼顾效率和可扩展性的解决方案。

Softmax

我们通过两阶段归约的并行计算策略，实现了高效且数值稳定的 Softmax 函数。第一阶段，`partial_softmax_kernel` 核函数将输入数据分块，每个线程块并行地计算其所负责数据的局部最大值和局部归一化和，并将结果存入中间数组。第二阶段，

`final_softmax_kernel` 核函数仅用一个线程块处理这些局部结果，先进行全局归约得到全局最大值和全局归一化和，然后所有线程再并行地利用这些全局值，对输入数组中的每个元素进行最终的 Softmax 计算。通过这种分而治之的并行归约方法，我们有效地解决了单次内核启动无法处理海量数据的问题，并利用了 GPU 架构中的共享内存进行块内归约，从而最大化了计算效率。

I/O性能优化

我们绕过C++ `iostream`的性能瓶颈，采用更底层的C风格I/O进行极致加速。输入端：用`freopen`重定向文件到标准输入，并结合非线性安全的`getchar_unlocked`实现快速读函数。这避免了`cin`的同步和格式化开销，通过逐字符读取直接构造数值，速度更快。输出端：摒弃逐项输出的`cout`，改为手动维护一个大容量字符缓冲区。所有结果先在内存中转换成字符串并存入该缓冲区，最后通过单次`fwrite`系统调用将全部内容一次性写入标准输出，极大减少了昂贵的系统调用次数，从而显著提升了整体效率。

总结

在APSP、Prefix Sum与Softmax三个算子的实现中，我们广泛采用了分块计算、共享内存优化、Warp级通信（如`__shfl_up`）、递归分治及两阶段归约等GPU并行优化技巧，显著提升数据局部性与计算效率，并在Prefix Sum与Softmax算子中采用C风格I/O与手动缓冲策略极致优化了数据读写性能。通过实践，我们深入理解了AMD HIP架构下线程调度、内存层次与同步机制的特点，掌握了如何规避全局内存瓶颈与数值溢出问题。

此次比赛的时间限制再加上大学课程繁忙对我们而言也是很大的挑战，紧张与压力下产生了一些心态上的问题，不过由于队友之间的互相帮助鼓励，我们及时调整心态，合理规划每日任务、设置阶段性目标，通过短会同步进度、快速排错，避免内耗；遇到瓶颈时主动查阅文档、请教导师，保持积极沟通与弹性应对。

在高压环境下，我们学会了用冷静代替焦虑，用协作化解孤立，将压力转化为高效执行的动力。团队协作中分工明确、反复调优，提升了工程化能力与性能敏感度，不仅强化了并行算法设计思维，也加深了对异构计算底层原理的认知，为未来高性能计算项目打下坚实基础。